F215 Digital Design — Tutorial 2

Canonical forms, K-maps, NAND/NOR universal gates, requirements-to-circuit design

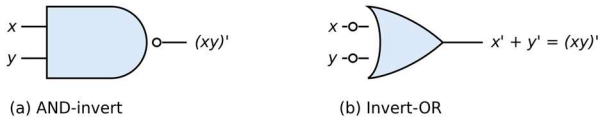
Question 1: NAND/NOR-Only Implementations (Universal Gates)

A fire-alarm circuit computes F = AB + A'C, where A = smoke detected, B = heat detected, C = CO confirmed.

(a) Draw a circuit for F using NOT, AND, and OR gates directly (any number of inputs per gate is fine).

A note on NAND gates

By De Morgan's law: $\overline{x \cdot y} = \overline{x} + \overline{y}$

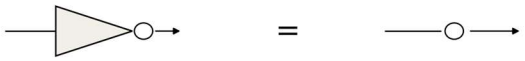


(a) AND-invert (b) Invert-OR

This means a NAND gate can be drawn two equivalent ways: as an AND gate with a bubble on its output (NOT of AND), or as an OR gate with bubbles on both inputs (OR of inverted inputs). Both symbols compute exactly the same function — which form you draw just depends on which is easier to match against the rest of your circuit.

A note on NOT gates

The same idea applies to a plain inverter: it can be drawn as the usual triangle-with-bubble symbol, or shorthand as just a bare bubble sitting on a wire. This is the form you'll use when bubble-pushing through a circuit — a bubble on a wire is a NOT gate.



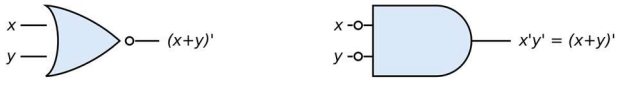
inverter symbol bare bubble on a wire

(b) Using this information, convert your circuit from (a) into an all-NAND implementation: take the circuit and add bubbles at the appropriate points so every gate becomes a NAND gate. (Hint: two bubbles on a wire cancel each other because NOT(NOT(x)) = x). How would you implement the NOT gate needed for the A' input using only a NAND gate?

(c) The CO sensor turns out to be unreliable on its own, so the design is revised to only trust a CO reading when it agrees with a high-temperature reading: F = AB + A'BC. Convert this to an all-2-input-NAND circuit (use as is without simplification).

A note on NOR gates

By the dual form of De Morgan's law: $\overline{x + y} = \overline{x} \cdot \overline{y}$



(a) OR-invert (b) Invert-AND

So a NOR gate can also be drawn two equivalent ways: as an OR gate with a bubble on its output, or as an AND gate with bubbles on both inputs.

(d) Now implement the original F = AB + A'C using only NOR gates. Before you start bubble-pushing on the AND-OR circuit from (a), it's worth asking whether there's a different algebraic form of F that might make this a much easier task.

Question 2: Canonical Forms, Minimization, and Choosing SOP vs. POS

Table 1

G	E	J	R
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

Table 2

G	E	J	R
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

(a) Write the canonical sum-of-products expression for R using minterms based on Table 1.

(b) Use a K-map to reduce this to a minimal sum of products (MSP).

(c) Find the minimal SOP for R using a K-map for Table 2.

(d) Now find the minimal POS for R using a K-map — group the 0s instead of the 1s, and use the maxterm rule to write each sum term.

(e) Compare the two forms from (c) and (d): how many gates does each need, and how many total gate-inputs? Which would you build, and why?

Question 3 — From Requirements to a Circuit: A Complete Design

A wireless sensor node transmits a 3-bit reading $D_2D_1D_0$ to a base station over a noisy link. To help the receiver catch single-bit transmission errors, the transmitter also sends one extra check bit P alongside the three data bits. The transmitter hardware is built so that, no matter what the 3-bit reading is, the total count of 1s among D_2, D_1, D_0, and P always comes out even.

(a) Build the truth table for P as a function of D_2, D_1, D_0.

(b) Derive the MSP for P. What is the gate count for implementing P? Can it be implemented with fewer gates?

(c) On the receiver side, how will you test if the received 4-bit value ($D_2D_1D_0P$) is correct or has exactly one bit corrupted? Design a circuit for this.

(d) Now we devise a different method to detect errors. The sensor encodes values in Gray code. The receiver circuit provides two outputs: the previous received value, and the current received value. Design a circuit that detects errors in the current value, under the assumption that readings change slowly, so consecutive readings can only be identical or neighbouring values.

(i) For each bit position, how would you compute whether that bit changed between the previous and current reading?

(ii) Given those three change-signals, how many can be 1 at once for a valid transition, and how many indicates an error? Design a circuit that outputs ERROR = 1 exactly when too many bits changed.